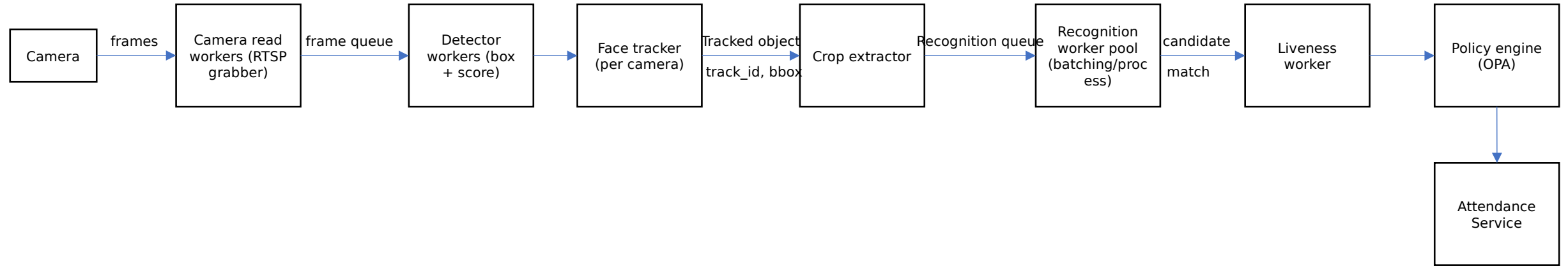


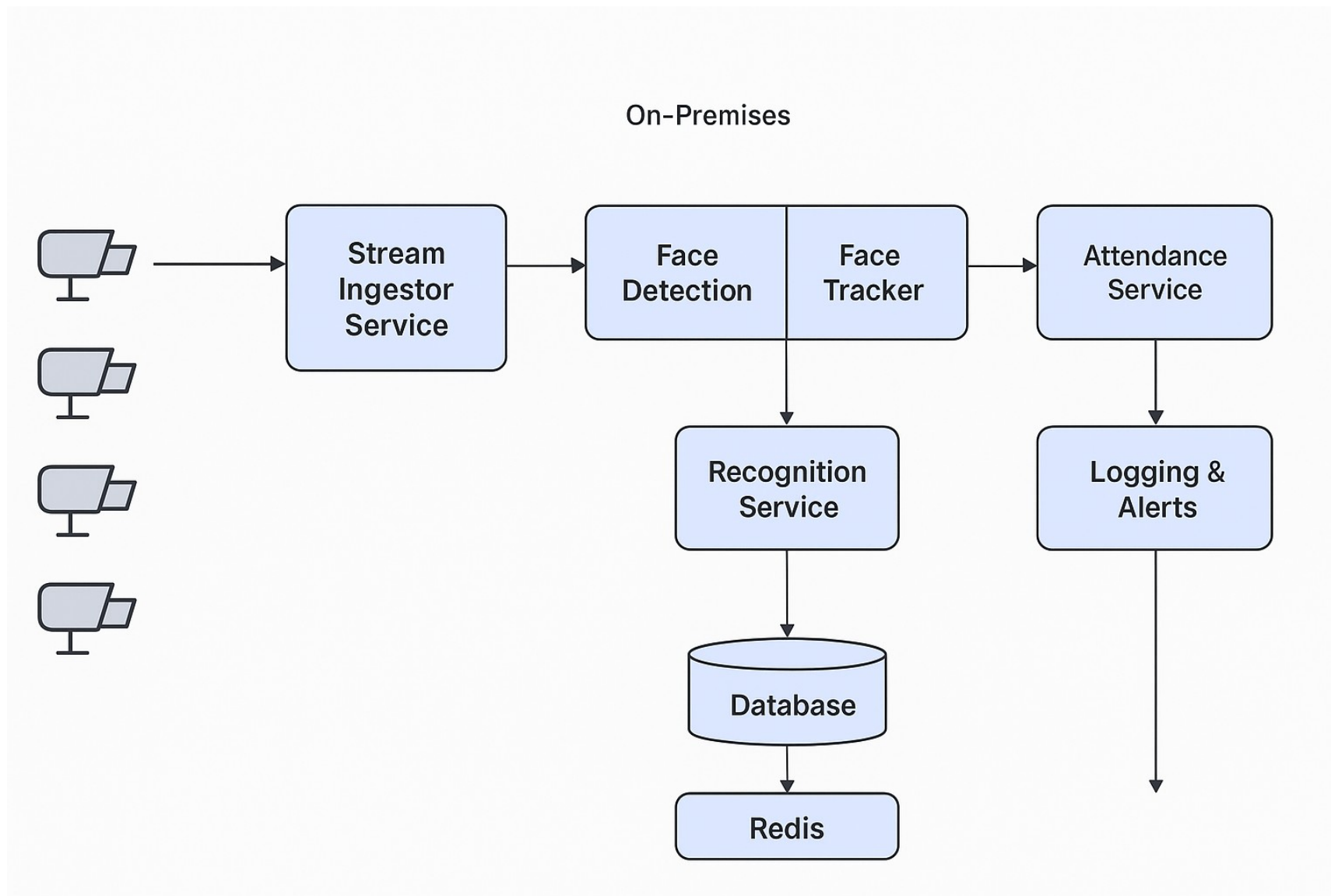
Project **Iris** (code name)

Software Architecture and Design Document

V1.0 (initial Draft)

Runtime Architecture & threading model □ Pipeline





Process layout (processes)

- **Stream Ingestor Manager** (process)
 - Spawns a lightweight RTSP grabber thread per channel.
 - Maintains per-channel frame queue (bounded).
 - Adjusts FPS throttle.
- **Detector Pool (process or threadpool)**
 - Runs face detection model (CPU).
 - (Haar cascades, MTCNN, BlazeFace, or MobileNet-SSD variants)
- **Per-Camera Tracker (in-process, single-threaded per camera)**
 - Runs ByteTrack/DeepSORT logic using detection boxes.
 - Produces stable track IDs.
- **Recognition Service (embedding extraction) (process)**
 - ArcFace/FaceNet with **ONNX Runtime (CPU EP)** or **OpenVINO**..
 - process one crop at a time.
 - Throughput: depending on model, **~10-30 embeddings/sec per CPU core** is typical.
- **Vector Index (FAISS)** (shared in-memory or separate process)
 - Exposes match API to recognition service.
- **Liveness Service** (process)
 - lightweight model on CPU (fast).
 - Called only for top candidates to conserve resources. - restrict liveness checks to only *top-N matches* instead of every crop.
- **OPA Policy Engine** (sidecar / service)
 - Evaluate rules: deny/allow based on user status, time, blacklist, etc.
- **Attendance Service (API + DB)**
 - Writes attendance events, triggers alerts/logs.
- **Logging & Alerting Engine**
 - Stores audit logs (append-only) and routes alerts to UIs/SMTP/SMS.
- **Redis**
 - Cache, short-term dedupe store, counters, feature flags, retry queues.

Concurrency model

- **RTSP Grabber:** 1 goroutine/thread per RTSP channel — minimal work, only decoding + pushing frames into queue.
- **Detector workers:** N CPU threads; size depends on CPU capacity. Non-blocking: We will use worker pools.
- **Tracker:** 1 per camera, single-threaded to simplify state. It consumes detections and emits events.
- **Recognition:** a bounded GPU worker pool (e.g., 1 worker per GPU) that supports batching. Use a small async queue and a timeout to trigger a batch.
- **Policy/Liveness:** separate pools to avoid blocking recognition pipeline.

Resource scheduling

- **Each RTSP decode:** ~small CPU usage; each 1080p (can go to 720p/480p) stream may require ~0.5–1 vCPU to decode (estimate varies).
- **Detector:** on CPU might handle ~5–10 FPS per core with light detectors.
- **Recognition GPU:** 1 GPU can serve tens to hundreds of crops/s depending on model and batching.

Note: Start and benchmark with the model performance.

Cache mechanism for fast ephemeral state, deduplication

Design Rationale:

- **Recent-seen / dedup window**
 - avoid multiple attendance events for the same employee from repeated frames/tracks.
 - Implementation example: store `recent_seen:{camera_id}:{user_id}` with small TTL (e.g., 10-120s). Presence = skip.
- **Track -> user binding cache**
 - while a track is active, cache `track_user:{camera_id}:{track_id} = {user_id, last_seen_ts}` TTL ~ track_timeout. Helps in reusing identity for same track.
- **User profile cache (fast read fallback)**
 - read-through cache for user profile/employee metadata so attendance service can proceed if Postgres is momentarily down. Example Cached items (Actual need to design during pipeline formation): `{user_id} -> {name, emp_code, dept, is_active, access_flags}`.
- **Retry queue / pending events**
 - Temporary storage for failed uploads/events that will be retried by background worker. Can be a Redis stream or list for reliability.
- **Alert dedupe / suppression**
 - Keep `alert_suppressed:{alert_key}` with TTL to avoid flooding (e.g., repeated spoof alerts suppressed for N seconds).

Flow:

- **Dedup & track binding flow**
 1. Tracker outputs track_id for detection.
 2. Recognition returns user_id with confidence.
 3. Worker checks `recent_seen:{camera}:{user}` — if exists → ignore (log dedup).
 4. Else write `recent_seen:{camera}:{user} = now` with TTL 60s and proceed to attendance logic.
 5. Also set `track_user:{camera}:{track} = {user_id, last_seen}` TTL track_timeout.
- **Fallback when DB down**
 - On recognition, fetch `user_profile:{user_id}` from Redis:
 - If present → proceed and write attendance event (mark source=cached).
 - If not present → record event to retry_queue and alert Deny Access User DB and log.
- **Alert suppression**
 - On spoof detection for user_id on camera, check `alert_suppressed:spoof:{camera}:{user}`:
 - If exists → do not send alert; otherwise send alert and set suppression key TTL 300s.

Failure handling, retries and DB-down behavior

Failure modes & handling mechanism

- **FAISS / Vector DB unavailable**
 - Recognition workers should fallback to a degraded mode: run embedding but mark `no_index` → push event to logging/alert and optionally store embedding to retry batch compare later.
 - Alert Deny Access Feature store or No match depending on logic.
- **Postgres user DB unavailable**
 - Use cached `user_profile:{user_id}` in Redis to proceed. If not cached, **do not block recognition** — create a provisional attendance event with `user_id` and `status=DB_LOOKUP_FAIL` and push to `retry_queue` for post-processing. Alert Deny Access User DB.
- **Liveness model failures / model crashes**
 - Fallback: mark liveness as unknown → policy decides (default deny). Log and alert Deny Access Spoof. Retry liveness for next frames for same track.
- **Network/IO issues**
 - RTSP disconnects: reconnect with exponential backoff (store last N frames).
 - Write to DB fails: write to Redis retry queue and background worker will reattempt.

Retry policy

Immediate: 3 attempts with small jitter (0.5s → 1s → 2s).

Queued: push to Redis stream `retry_queue` with `attempt_count`. Background retry worker processes exponential backoff: `attempt_count * base_delay` up to max attempts (e.g., 5).

On final failure, record permanent audit log and raise high-severity alert.

Alert classification & mapping to actions

Alert Key	Trigger	Severity	Action
DenyAccess_NoFace	No face crop found at checkpoint / repeated	Medium	Deny access, log, notify guard UI
DenyAccess_FeatureStore	FAISS/index failure	High	Deny or fallback; alert Ops
Alert_Security_Repeated	Repeated failed matches or tampering	High	Notify security + create incident
DenyAccess_NoMatch	Face matched below confidence & policy denies	Medium	Deny access, show message, log
Alert_Security_Spoof	Liveness failed (spoof)	Critical	Immediate security alert, lock gate if integrated
DenyAccess_UserDB	User DB unreachable for matched ID and no cache	High	Deny/Quarantine, raise ops alert
DenyAccess_Timeout	Decision timeout (pipeline stalled)	Medium/High	Deny, log, notify Ops

Alerts sink:

- **Monitoring UI:** show active alerts & status
- **Security Dashboard:** show immediate security incidents
- **Audit logs:** append-only event store for compliance

Policy evaluation flow (OPA integration)

Flow when a candidate is recognized:

- Recognition returns {user_id, conf}.
- Liveness result: pass/fail/unknown.
- Query OPA with data: {user_profile, camera_meta, time, conf, liveness}.
- OPA returns decision: {allow: true/false, reason, actions}.
- Attendance service acts: create attendance event if allowed; otherwise deny and send alert according to reason.

Make decisions readable/auditable: keep the OPA input & output in the audit log for incident investigations.

CPU Sizing Evaluation

Workload Assumptions:

- **Video resolution:** 720p (or 640p low priority)
- **Frame rate processed:** ~5 fps (not full 30 fps)
- **Face density:** 1-3 faces per frame (typical entrance scenarios). – Need to evaluate at actual
- **Detection:** lightweight model (BlazeFace / RetinaFace-CPU)
- **Recognition:** ArcFace/FaceNet CPU via ONNX Runtime/OpenVINO
- **Vector DB:** FAISS (CPU, in-memory index)
- **Liveness:** lightweight model (MediaPipe / small CNN)
- **Tracker:** ByteTrack/DeepSORT (low CPU overhead)

Rough CPU Performance Metrics:

- **Decode RTSP (720p):** ~0.3 vCPU per stream (vs 0.5 for 1080p).
- **Face detection (BlazeFace/RetinaFace-CPU):** 1 vCPU can process ~15 fps (vs 10 at 1080p).
- **Recognition (ArcFace/FaceNet CPU):** 1 vCPU → ~12-15 crops/sec (vs 8-12 at 1080p).
- **Liveness (light CNN/MediaPipe):** 1 vCPU → ~12 crops/sec.
- **FAISS search:** negligible (~<5 ms per query with 10-20k templates).

Deployment Sizing

- **Small (≤ 10 cameras @ 5 fps) □ Saturday Demo with this**
 - **Cores:** 6-8 vCPU
 - **RAM:** 16 GB
- **Medium (25-30 cameras)**
 - **Cores:** 12-14 vCPU (instead of 16)
 - **RAM:** 24-32 GB

Development Task breakdown

Team- Video+ AI/ML

Epic 1 — Camera & Stream Handling

- **Story 1.1:** Implement NVR integration (RTSP pull, config per camera with name/location). The camera configuration need to fetch from the database.
- **Story 1.2:** Build Camera Handling Service (threaded workers for decoding).
- **Story 1.3:** Add frame throttling (configurable fps per camera).
- **Story 1.4:** Buffer frames (local queue) & retry mechanism.
- **Story 1.5:** Health check API (camera online/offline status).

Epic 2 — Stream Ingestor (Detection + Tracking)

- **Story 2.1:** Integrate face detection (BlazeFace/RetinaFace-CPU).
- **Story 2.2:** Implement tracker (DeepSORT/ByteTrack).
- **Story 2.3:** Deduplicate faces (per-camera time window).
- **Story 2.4:** Extract cropped faces → send to Recognition service.
- **Story 2.5:** Redis integration to buffer detection output.

Development Task breakdown

Team- Video+ AI/ML

Epic 3 — Recognition Service

- **Story 3.1:** Implement embedding extraction (ArcFace/FaceNet, ONNX Runtime CPU).
- **Story 3.2:** Store embeddings in FAISS (CPU index).
- **Story 3.3:** Query FAISS for top-K matches, return confidence.
- **Story 3.4:** Redis cache for recent embeddings (avoid recompute in short window).
- **Story 3.5:** Handle low-confidence / no-match → send alert event.

Epic 4 — Liveness Detection

- **Story 4.1:** Integrate lightweight liveness (MediaPipe/anti-spoof CNN or option to find something else which work on CPU).
- **Story 4.2:** Run liveness only on recognized candidates (optimize CPU).
- **Story 4.3:** Fail event → push “spoof alert” to Alerts Service.

Development Task breakdown

Team- Core Server logic

Epic 5 — Attendance Service

- **Story 5.1:** Define attendance event schema (IN/OUT, timestamp, camera, employee_id).
- **Story 5.2:** Implement deduplication (per employee within N-minute window).
- **Story 5.3:** Apply business rules (shift mapping, IN/OUT detection).
- **Story 5.4:** Integrate with OPA/Keycloak for access policy evaluation.
- **Story 5.5:** Persist events to Postgres DB.

Epic 6 — Alerts & Logging

- **Story 6.1:** Define alert categories (No face, No match, Spoof, Repeated attempts, DB down).
- **Story 6.2:** Build Alert Service (publish/subscribe model).
- **Story 6.3:** Store alerts in Audit Log DB.
- **Story 6.4:** Integrate with monitoring (Prometheus/Grafana).
- **Story 6.5:** Expose API for dashboard notifications.

Development Task breakdown

Team- Core Server logic

Epic 7 — Caching & Reliability

- **Story 7.1:** Use Redis for:
 - recent embeddings (avoid recompute),
 - deduplication state (employee recently seen),
 - frame queue (temporary if recognition is slow).
- **Story 7.2:** Implement retry queue for failed DB writes.
- **Story 7.3:** Implement agent-to-server message buffering for outages.

Epic 8 — Admin & Config Management

Team- Frontend

- **Story 8.1:** Camera configuration UI (add/edit/delete, assign location) – Implement onvif client to discover camera and auto add feature.
- **Story 8.2:** Employee CRUD (add/edit, upload enrollment images).
- **Story 8.3:** Sync embeddings into FAISS index after enrollment.
- **Story 8.4:** Shift & policy management UI.

Development Task breakdown

Team- DevOps/Combine

Epic 9 — Deployment & Ops - Optional low priority

- **Story 9.1:** Dockerize each service (camera handler, ingestor, recognition, attendance, alerts).
- **Story 9.2:** Write docker-compose for single-server deployment.
- **Story 9.3:** Monitoring stack (Prometheus + Grafana).
- **Story 9.4:** Automated backups (Postgres, Redis, FAISS index).

Epic 10 — Camera Simulator Development

Team- Test/Combine

- **Story 10.1:** Implement simple RTSP simulator using prerecorded video files (FFmpeg -stream_loop + RTSP server).
- **Story 10.2:** Build simulator module to generate synthetic faces (random images, shuffle employees).
- **Story 10.3:** Add configuration for multiple virtual cameras (each with camera_id, location).
- **Story 10.4:** Implement "no face" scenario simulator (empty corridor/scene).
- **Story 10.5:** Implement "spoof" scenario (printout/photo attack clips).
- **Story 10.6:** Add time-based event playback (simulate shift change, crowd entry).
- **Story 10.7:** Integrate simulator into CI/CD pipeline (auto-run test streams on each build). – **Optional (Later)**
- **Story 10.8:** Web UI toggle: choose between live NVR camera vs simulated camera in config. – **Optional (Later)**